

시큐어 코딩 과제 보고서

- 시큐어 코딩 과제 보고서
 - Tiny Second-hand Shopping Platform
- 1. 요구사항 분석
 - 1.1 프로젝트 목표
 - 1.2 이해관계자(Actor) 정의
 - 1.3 기능 요구사항 (Functional Requirements)
 - 1.4 비기능 요구사항 (Non-Functional Requirements)
 - 1.5 보안 요구사항 (Security Requirements)
 - 1.6 제약 사항 및 가정
 - 1.7 다음 단계
- 2. 시스템 설계
 - 2.1 아키텍처 개요
 - 2.2 기술 스택 및 선정 근거
 - 2.3 페이지(화면) 설계
 - 2.4 데이터베이스 설계
 - 2.5 라우트 / 이벤트 설계 (요약)
 - 2.6 보안 설계 (SR → 구현 방식 매핑)
 - 2.7 위협 모델 (요약, 자산 기준)
 - 2.8 다음 단계
- 3. 시스템 구현
 - 3.1 구현 개요
 - 3.2 모듈 구조
 - 3.3 공통 보안 통제 (전역)
 - 3.4 기능별 구현 요점 (SR 실현)
 - 3.5 데이터 무결성 (DB 제약)
- 4. 테스트 (체크리스트 & 테스트 케이스)
 - 4.1 자동화 테스트 개요
 - 4.2 보안 체크리스트 (슬라이드 31쪽 기준)
 - 4.3 대표 공격 시나리오 테스트 (발체)
 - 4.4 실행 결과
 - 4.5 정적 보안 분석 (SAST)
- 5. 유지보수
 - 5.1 의존성 취약점 관리 (실제 수행)

- 5.2 시크릿 관리
- 5.3 운영 배포 설정
- 5.4 로깅 & 모니터링
- 5.5 백업 & 데이터 관리
- 5.6 사고 대응 절차 (요약)
- 5.7 유지보수 관점의 개선 여지 (알려진 한계)
- 6. 발견한 보안 약점과 수정 (Before / After)
 - 6.1 SQL Injection — 인증 우회 (SR-02)
 - 6.2 비밀번호 평문 저장 (SR-01)
 - 6.3 권한 상승 — Mass Assignment (SR-11)
 - 6.4 저장형 XSS — 상품/소개글/채팅 (SR-03)
 - 6.5 IDOR — 타인 상품 수정/삭제 (SR-05)
 - 6.6 송금 경쟁 조건 — 더블스펜딩 (SR-08)
 - 6.7 잘못된 송금 입력 — 음수/자기송금 (SR-08)
 - 6.8 악성 파일 업로드 — SVG/위장 이미지 (SR-13)
 - 6.9 신고 남용 — 자기/중복 신고 (SR-12)
 - 6.10 CSRF — 상태변경 위조 (SR-04)
 - 6.11 세션·전송 보안 (SR-06, SR-16)
 - 6.12 정보 노출 & 무차별 대입 (SR-09, SR-10, SR-15)
 - 6.13 의존성 취약점 (유지보수, SR 상시)
 - 6.14 신고 기반 관리자 계정 잠금 — DoS (독립 감사에서 발견)
 - 6.15 워커 풀 다중 리뷰 반영 (2차 하드닝)
 - 6.16 방어적 하드닝 (테스터 관점 선제 조치)
 - 6.17 세션 고정(Session Fixation) — PM 리뷰 패널이 발견
 - 요약 표
- 7. AI 도구 활용
 - 7.1 활용 방식
 - 7.2 다중 에이전트 적대적 리뷰 (핵심)
 - 7.3 AI가 실제로 잡아낸 취약점 (사람이 놓친 것)
 - 7.4 환각 방지 원칙
 - 7.5 결론

시큐어 코딩 과제 보고서

Tiny Second-hand Shopping Platform

보안 약점을 최소화한 중고거래 플랫폼 개발 — 개발 전 과정 보고서

항목	내용
과정	WhiteHat School · Secure Coding
반	(○○반 기입)
이름	(이름 기입)
연락처	(전화번호 뒷자리 기입)
GitHub	https://github.com/choewonwoo1817/whs4th/tree/main/secure-coding
제출일	2026-07-__

기술 스택: Python 3.12 · Flask · Flask-SocketIO · SQLite **핵심:** 요구사항 분석 → 시스템 설계 → 구현 → 테스트(85개 통과) → 유지보수, 전 과정에 걸친 시큐어 코딩 적용

1. 요구사항 분석

Tiny Second-hand Shopping Platform — 시큐어 코딩 과제 작성자: (이름/반/연락처 기입) · 문서 버전 v0.1

1.1 프로젝트 목표

“중고거래가 가능한, 안전한 웹 플랫폼”을 개발한다.

단순히 동작하는 것을 넘어, **개발 전 과정에 걸쳐 보안을 고려(시큐어 소프트웨어 공학)** 하여 설계 단계부터 보안 약점을 선제적으로 제거하는 것을 최우선 목표로 한다.

- 1차 목표: 중고거래 플랫폼의 최소 기능 요구사항을 모두 충족한다.
- 2차 목표: 각 기능에서 발생 가능한 보안 약점을 식별하고 안전하게 구현한다.
- 3차 목표: 발견한 약점과 수정 내용을 문서로 남겨 재현·검증 가능하게 한다.

1.2 이해관계자(Actor) 정의

Actor	설명	신뢰 수준
비로그인 방문자(Guest)	가입/로그인 전 사용자. 상품 목록·상세 열람만 가능	낮음 (신뢰 불가)
일반 회원(User)	가입·로그인한 사용자. 상품 등록/거래/채팅/신고/송금	중간 (부분 신뢰)
관리자(Admin)	플랫폼 전체를 관리. 유저/상품/신고 처리	높음
공격자(Attacker)	위 어떤 역할로도 위장 가능한 악의적 주체	적대적

보안 설계 원칙: **모든 입력은 신뢰하지 않는다**(Guest·User·Admin 요청 모두 검증 대상). 특히 “로그인했으니 신뢰”가 아니라, “이 사용자가 이 자원에 대해 이 행위를 할 권한이 있는가”를 매 요청 검증한다.

1.3 기능 요구사항 (Functional Requirements)

슬라이드 22·25·26쪽의 요구 기능을 기반으로 도출. 각 요구사항에 ID를 부여해 이후 설계·구현·테스트에서 추적한다.

(1) 유저 관리

ID	요구사항	비고
FR-01	회원가입 (아이디, 비밀번호, 계정명)	아이디 중복 불가
FR-02	로그인 / 로그아웃	세션 기반 인증
FR-03	사용자 프로필 조회	타 사용자 프로필 열람
FR-04	마이페이지 — 소개글 수정	본인만
FR-05	마이페이지 — 비밀번호 변경	본인만, 현재 비밀번호 확인
FR-06	유저 정보는 데이터베이스로 관리	

(2) 상품 관리

ID	요구사항	비고
FR-10	상품 등록 (상품명, 가격, 설명, 사진)	로그인 사용자만
FR-11	내가 등록한 상품 조회·수정·삭제	소유자만
FR-12	전체 상품 목록 조회 (이름만 노출)	누구나
FR-13	상품 상세 페이지 (클릭 시 상세 정보)	누구나
FR-14	상품 검색	이름/키워드
FR-15	상품 정보는 데이터베이스로 관리	

(3) 사용자 간 소통

ID	요구사항	비고
FR-20	실시간 전체 채팅	전체 유저 대상
FR-21	1:1 채팅	두 사용자 간

(4) 악성 유저/상품 필터링

ID	요구사항	비고
FR-30	불량 유저/상품 신고 (사유 작성)	로그인 사용자만
FR-31	일정 횟수 이상 신고된 상품 자동 차단	임계치 기반
FR-32	일정 횟수 이상 신고된 유저 자동 휴면 전환	임계치 기반

(5) 송금

ID	요구사항	비고
FR-40	사용자 간 송금 (포인트/잔액 이체)	잔액 개념 도입 필요
FR-41	거래 내역 기록	무결성 보장

(6) 관리자

ID	요구사항	비고
FR-50	관리자 페이지 — 전체 유저/상품/신고 관리	Admin 전용
FR-51	상품 강제 삭제 / 유저 휴면·복구 처리	Admin 전용

요구사항 변경/추가 내역 (과제 허용 범위, 슬라이드 35쪽 근거): - 송금(FR-40) 구현을 위해 **사용자 잔액(balance)·거래내역(transfers)** 개념을 추가한다. (송금은 반드시 “얼마를, 누구에게서 누구로” 이동하는 상태 변화이므로 잔액·거래 테이블 없이는 논리적으로 성립 불가.) - 신고 임계치 자동 처리(FR-31/32)를 위해 **신고 집계·상태(is_blocked, is_dormant)** 필드를 추가한다.

1.4 비기능 요구사항 (Non-Functional Requirements)

슬라이드 22쪽 “필요한 비기능적 요소: 보안 / 디자인 / 서버 속도”.

ID	항목	요구사항
NFR-01	보안	후술하는 보안 요구사항(SR)을 모두 충족. 최우선 순위 .
NFR-02	사용성/디자인	페이지 이동이 직관적이고 폼 오류 메시지가 명확
NFR-03	성능	목록/검색 조회가 즉시 응답 (SQLite 인덱스 활용)
NFR-04	이식성	Linux(Ubuntu)/miniconda 환경에서 README만으로 실행 가능
NFR-05	유지보수성	기능별 모듈(Blueprint) 분리, 입력검증 로직 중앙화

1.5 보안 요구사항 (Security Requirements)

시큐어 코딩 과제의 핵심. **각 기능에서 실제로 발생 가능한 위협**을 기준으로 도출했으며, KISA 소프트웨어 보안약점 7개 유형 및 OWASP Top 10과 매핑한다.

ID	보안 요구사항	대응 위협(공격 시나리오)	KISA 유형	관련 FR
SR-01	비밀번호는 평문 저장 금지, salt 적용 해시 (bcrypt/Argon2)로 저장	DB 유출 시 전 계정 탈취	보안기능	FR-01,05
SR-02	모든 DB 쿼리는 파라미터 바인딩 사용	SQL Injection으로 인증 우회·데이터 탈취	입력값 검증	전체
SR-03	사용자 입력의 출력 시 이스케이프(자동 이스케이프 유지)	저장형 XSS(상품 설명·소개글·채팅)	입력값 검증	FR-04,10,20,21
SR-04	상태 변경 요청(POST)에 CSRF 토큰 검증	CSRF로 비밀번호 변경·송금 강제	보안기능	FR-05,10,40
SR-05	자원 접근 시 소유자/권한 확인	IDOR — 남의 상품 수정/삭제, 남의 마이페이지 변경	캡슐화	FR-04,05,11,51
SR-06	세션 쿠키에 HttpOnly·Secure·SameSite 적용, 로그인 시 세션 재발급	세션 탈취·세션 고정	보안기능	FR-02
SR-07	서버 측 입력값 검증(길이·형식·범위)	클라이언트 검증 우회 입력	입력값 검증	전체 품
SR-08	송금 금액 검증(양수·잔액 이내)과 원자적 처리	음수 송금·잔액 초과·동시요청 더블스펜딩	시간 및 상태	FR-40
SR-09	오류 메시지에 내부 정보(스택트레이스·쿼리) 노출 금지	정보 노출을 통한 공격 정보 수집	에러 처리	전체
SR-10	로그인·신고·채팅 등에 Rate Limiting 적용	무차별 대입·스팸·신고 남용	시간 및 상태	FR-02,20,30
SR-11	관리자 기능은 Admin 권한 세션만 접근	권한 상승으로 관리자 기능 탈취	캡슐화	FR-50,51
SR-12	신고는 본인 신고 불가·중복 신고 제한	자기신고/반복신고로 임계치 악용	시간 및 상태	FR-30,31,32
SR-13	업로드 이미지의 확장자·MIME·크기 검증	악성 파일 업로드(웹셸)	입력값 검증	FR-10
SR-14	파일/자원 식별자 직접 노출 대신 검증 후 접근	경로 조작(Path Traversal)	입력값 검증	FR-10,13

ID	보안 요구사항	대응 위협(공격 시나리오)	KISA 유형	관련 FR
SR-15	로그인 연속 실패 시 점진적 지연/일시 잠금	무차별 대입 가속 (rate limit과 별개)	시간 및 상태	FR-02
SR-16	통신 구간 TLS/WSS 암호화	중간자 도청(자격증명·채팅 내용)	보안기능	FR-02,20,21

위 SR 목록은 “시스템 설계” 문서의 보안 설계 및 “테스트” 문서의 체크리스트로 1:1 이어진다.

1.6 제약 사항 및 가정

- 실행 환경: Ubuntu(Linux) + miniconda(Python) + ngrok 외부 노출 (슬라이드 16쪽).
- 프레임워크: Python 기반 웹(Flask 계열) — 실시간 채팅 요구로 WebSocket 지원 필요.
- 데이터 저장소: 과제 규모에 적합한 파일 기반 RDBMS(SQLite).
- 결제/송금은 실제 금융망이 아닌 **플랫폼 내부 포인트(잔액)** 이동으로 구현한다.
- 인증은 외부 IdP 없이 자체 세션 기반으로 구현한다.

1.7 다음 단계

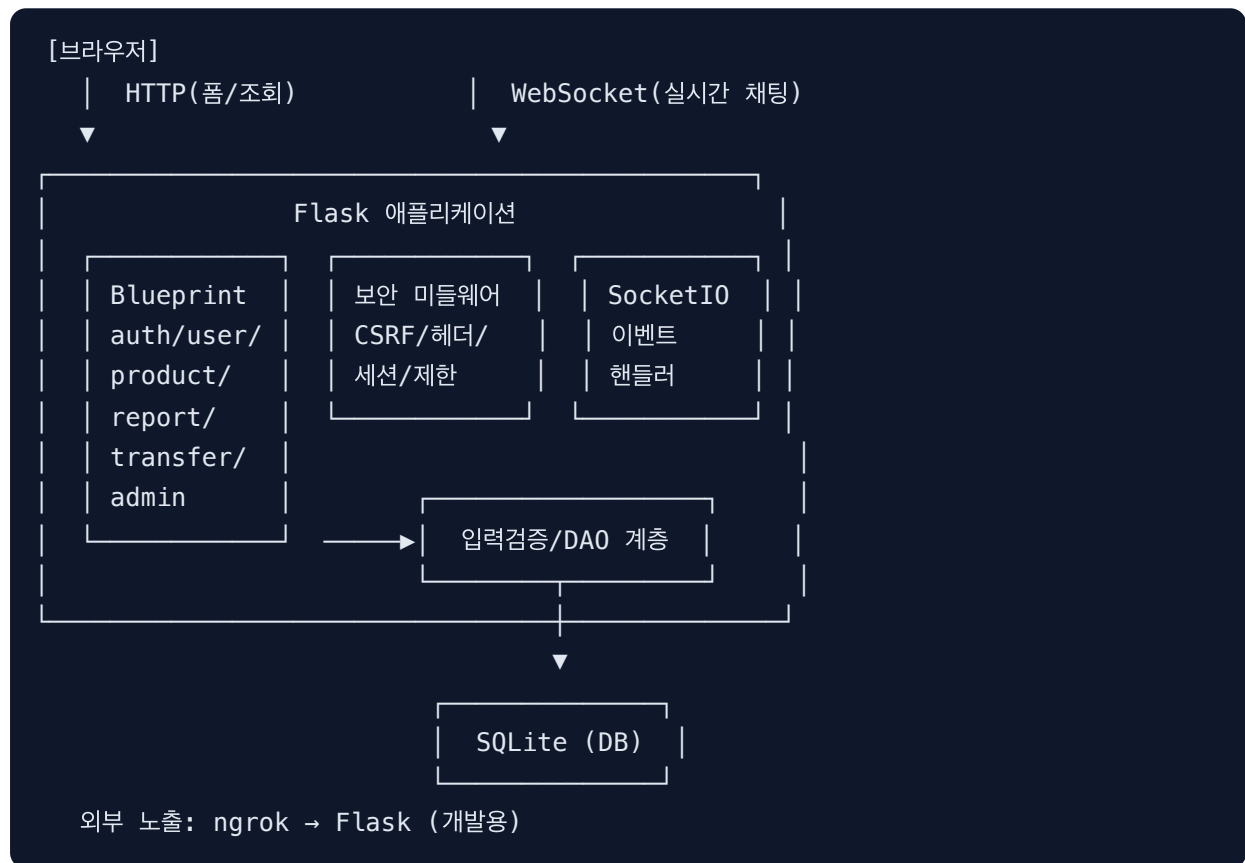
본 요구사항(FR/NFR/SR)을 입력으로 하여 → **02_시스템설계.md** 에서 아키텍처, 페이지, DB 스키마, 라우트, 그리고 각 SR을 어떻게 기술적으로 구현할지 설계한다.

2. 시스템 설계

Tiny Second-hand Shopping Platform — 시큐어 코딩 과제 입력 문서: 01_요구 사항분석.md · 문서 버전 v0.1

2.1 아키텍처 개요

전형적인 서버 렌더링(MPA) 웹 아키텍처 + 실시간 채팅을 위한 WebSocket 계층.



- **관심사 분리:** 기능별 Blueprint로 라우트를 나누고, DB 접근은 DAO(데이터 접근 계층)로 중앙화한다. → SQL 인젝션 방지(파라미터 바인딩)와 입력검증을 한 곳에서 강제하기 위함(SR-02, SR-07).
- **보안 미들웨어:** 모든 응답에 보안 헤더, 모든 상태변경 요청에 CSRF 검증, 세션 설정을 전역 적용.

2.2 기술 스택 및 선정 근거

구분	선택	근거
언어/런타임	Python 3.12 (miniconda)	슬라이드 환경 세팅 지침(miniconda) 준수
웹 프레임워크	Flask	경량, 실시간 채팅 (SocketIO)·ngrok·miniconda 조합에 적합. (슬라이드 베이스 레포도 Flask 로 추정되나 실물 미확인)
실시간 통신	Flask-SocketIO	FR-20/21 실시간 채팅(WebSocket) 요구
템플릿	Jinja2	자동 이스케이프 기본 제공 → 저장형 XSS 1차 방어(SR-03)
CSRF/폼	Flask-WTF	CSRF 토큰 발급·검증 표준화(SR-04)
비밀번호 해시	argon2-cffi (또는 bcrypt)	salt 자동 적용 강력 해시(SR-01)
Rate Limiting	Flask-Limiter	무차별 대입·스팸·신고 남용 제한(SR-10)
DB	SQLite	과제 규모 적합, 파일 기반, 파라미터 바인딩 지원(SR-02)
외부 노출	ngrok	슬라이드 지침, 개발용 HTTPS 터널

보안 관점의 스택 선택: 프레임워크 기본 기능(Jinja2 자동 이스케이프, Flask 세션 서명, Flask-WTF CSRF)을 최대한 활용하여 “직접 구현하다 실수하는” 표면을 줄인다.

2.3 페이지(화면) 설계

슬라이드 27쪽 기준 + 요구 기능 반영.

페이지	경로(예상)	접근 권한	대응 FR
기본/홈	/	누구나	FR-12
회원가입	/register	Guest	FR-01
로그인	/login	Guest	FR-02
전체 상품 목록 + 전체 채팅	/products , /chat	누구나(채팅은 로그인)	FR-12, FR-20
상품 상세	/products/<id>	누구나	FR-13

페이지	경로(예상)	접근 권한	대응 FR
상품 등록	/products/new	User	FR-10
내 상품 관리(수정/삭제)	/products/<id>/edit	소유자	FR-11
상품 검색	/search	누구나	FR-14
프로필 조회	/users/<id>	누구나	FR-03
마이페이지	/mypage	본인	FR-04, FR-05
1:1 채팅	/chat/<user_id>	로그인 당사자	FR-21
신고	/report	User	FR-30
송금	/transfer	User	FR-40
관리자	/admin	Admin	FR-50, FR-51

2.4 데이터베이스 설계

슬라이드 28쪽의 3개 엔티티(사용자/상품/신고)를 기준으로 하되, 송금·신고 자동처리·보안(해시/상태) 요구를 반영해 확장한다. **확장 컬럼은 근거를 함께 명시.**

user (사용자)

컬럼	타입	설명	보안 근거
id	TEXT(PK)	내부 식별자(UUID)	추측 가능한 순번 노출 방지
username	TEXT UNIQUE	로그인 아이디	중복 불가(FR-01)
display_name	TEXT	계정명	
password_hash	TEXT	argon2 해시	평문 저장 금지(SR-01)
bio	TEXT	소개글	출력 시 이스케이프(SR-03)
balance	INTEGER	잔액(포인트)	송금(FR-40), 음수 불가(SR-08)
is_admin	INTEGER(0/1)	관리자 여부	권한 판정(SR-11)
is_dormant	INTEGER(0/1)	휴면 여부	신고 임계치 처리(FR-32)
report_count	INTEGER	누적 피신고 수	임계치 판정(FR-32)
created_at	TEXT	가입 시각	감사 로그

product (상품)

컬럼	타입	설명	보안 근거
id	TEXT(PK)	상품 식별자(UUID)	열거(enumeration) 완화 — 접근제어 자체는 SR-05로 별도 강제(UUID는 IDOR 방어 아님)
title	TEXT	상품명	입력검증(SR-07)
description	TEXT	설명	이스케이프(SR-03)
price	INTEGER	가격	양수·범위 검증(SR-07)
image_path	TEXT	사진 경로	확장자/MIME 검증(SR-13,14)
seller_id	TEXT(FK→user.id)	판매자	소유자 확인(SR-05)
is_blocked	INTEGER(0/1)	차단 여부	신고 임계치(FR-31)
report_count	INTEGER	누적 피신고 수	임계치 판정(FR-31)
created_at	TEXT	등록 시각	

report (신고)

컬럼	타입	설명	보안 근거
id	TEXT(PK)	신고 식별자	
reporter_id	TEXT(FK→user.id)	신고자	본인 신고 차단(SR-12)
target_type	TEXT	‘user’ 또는 ‘product’	
target_id	TEXT	대상 식별자	존재 검증(SR-07)
reason	TEXT	신고 사유	이스케이프·길이 검증(SR-03,07)
created_at	TEXT	신고 시각	
—	UNIQUE(reporter_id, target_type, target_id)	중복 신고 방지	임계치 악용 차단(SR-12)

transfers (거래/송금 내역) — 신규

⚠ 테이블명은 `transaction` 이 아닌 `transfers` 로 한다. `transaction` 은 SQL 예약어라 식별자로 쓰면 구문 오류·이스케이프가 필요하기 때문.

컬럼	타입	설명	보안 근거
id	TEXT(PK)	거래 식별자	
sender_id	TEXT(FK→user.id)	보내는 사람	본인만 출금(SR-05,08)
receiver_id	TEXT(FK→user.id)	받는 사람	존재 검증
amount	INTEGER	금액(>0)	음수/0 차단(SR-08)
created_at	TEXT	거래 시각	감사·무결성(FR-41)

message (채팅 메시지) — 신규(1:1 이력 저장용)

컬럼	타입	설명
id	TEXT(PK)	메시지 식별자
sender_id	TEXT	보낸 사람
receiver_id	TEXT NULL	받는 사람(전체 채팅이면 NULL)
content	TEXT	내용(이스케이프 길이 검증)
created_at	TEXT	시각

인덱스: `product(seller_id)` , `product(title)` (검색),
`report(target_type,target_id)` , `transfers(sender_id)` ,
`message(receiver_id)` — 성능(NFR-03) 및 임계치 집계 효율.

무결성 제약 (DB가 최후 방어선, SR-08·데이터 무결성): `PRAGMA`
`foreign_keys=ON` 으로 모든 FK 강제, 필수 컬럼 `NOT NULL` , `user.balance`
`CHECK(balance >= 0)` , `transfers.amount CHECK(amount > 0)` , `report` 의
`UNIQUE(reporter_id, target_type, target_id)` . → 애플리케이션 검증이
실패해도 DB 제약이 잘못된 상태를 거부한다.

2.5 라우트 / 이벤트 설계 (요약)

메서드	경로	기능	권한/검증
GET/POST	<code>/register</code>	회원가입	중복 검사, 입력검증, 해시 저장
GET/POST	<code>/login</code>	로그인	Rate limit, 세션 재발급

메서드	경로	기능	권한/검증
POST	/logout	로그아웃	CSRF, 세션 파기
GET/POST	/mypage	소개글·비번 변경	로그인, 본인, 현재 비번 확인, CSRF
GET	/users/<id>	프로필 조회	존재 검증, 이스케이프
GET	/products	목록	차단 상품 제외
GET/POST	/products/new	상품 등록	로그인, 입력·이미지 검증, CSRF
GET	/products/<id>	상세	차단/존재 검증
GET/POST	/products/<id>/edit	수정(폼 GET·저장 POST)	소유자 검증(SR-05), CSRF
POST	/products/<id>/delete	삭제	소유자 검증(SR-05), CSRF (GET 삭제 금지 — 링크 크롤링·CSRF로 삭제 유발 방지)
GET	/search	검색	파라미터 바인딩(SR-02)
GET	/chats	내 1:1 대화 목록(구매자·판매자가 대화를 찾는 곳)	로그인, 본인 대화만
GET	/chat/<user_id>	1:1 채팅방·이력	로그인, 본인 참여 대화만 조회(sender_id=나 OR receiver_id=나 강제, SR-05)
POST	/report	신고	로그인, 본인신고·중복 차단, 임계치 처리
POST	/transfer	송금	로그인, 금액검증, 원자적 트랜잭션(SR-08), CSRF
GET/POST	/admin/...	관리	Admin 전용(SR-11), CSRF
WS event	send_message	채팅 송신	인증 확인, 길이검증, 앱 수준 Rate limit(SR-10; Flask-Limiter 미적용)

2.6 보안 설계 (SR → 구현 방식 매핑)

요구사항의 보안 요구(SR)를 어떤 기술로 구현할지 확정한다. (구현·수정 서사가 여기서 이어짐)

SR	구현 설계
SR-01 비밀번호 해시	argon2-cffi PasswordHasher 로 해시/검증. 평문·단순 MD5/SHA 금지
SR-02 SQLi 방지	모든 쿼리 ? 파라미터 바인딩. 문자열 포매팅 쿼리 전면 금지. DAO 계층으로 통제
SR-03 XSS 방지	서버 렌더링 페이지는 Jinja2 자동 이스케이프 유지 (safe 금지). 실시간 채팅은 클라이언트 JS가 DOM에 삽입하므로 Jinja가 보호하지 못함 → JS에서 textContent 로 삽입(또는 서버가 이스케이프 후 브로드캐스트). 저장 전 길이·문자 검증 병행
SR-04 CSRF	Flask-WTF CSRFProtect 전역 적용. 모든 POST 폼에 토큰
SR-05 IDOR/권한	자원 조작 전 소유자 일치 확인(<code>product.seller_id == session['user_id']</code>). 실패 시 403
SR-06 세션	서버 측 세션 저장(Flask-Session; 클라이언트 쿠키엔 세션 ID만) → 휴면 전환·로그아웃 시 서버가 세션을 즉시 무효화 (FR-32 보장). SESSION_COOKIE_HTTPONLY=True , SAMESITE=Lax , 로그인 성공 시 세션 재발급. SECURE 는 HTTPS(ngrok)에서 활성화 — 로컬 http 테스트 시 미전송되므로 환경변수 토글. PERMANENT_SESSION_LIFETIME 로 유효/절대 타임아웃
SR-07 입력검증	서버 측 검증 유틸(길이/정규식/범위). 클라이언트 검증에 의존하지 않음
SR-08 송금 무결성	BEGIN IMMEDIATE 트랜잭션 내에서 잔액 확인·차감·증가를 원자적 수행. 금액>0·정수·잔액≥금액 검증
SR-09 에러 처리	전역 에러 핸들러로 사용자에게 일반 메시지, 상세는 서버 로그로만
SR-10 Rate Limit	HTTP 요청(로그인·신고)은 Flask-Limiter로 제한. 채팅은 SocketIO 이벤트라 Flask-Limiter가 적용되지 않으므로, 세션별 최근 전송 시각을 추적하는 앱 수준 제한을 별도 구현
SR-11 관리자 권한	@admin_required 데코레이터로 is_admin 확인
SR-12 신고 악용 방지	본인 신고 거부, UNIQUE 제약으로 중복 신고 차단, 임계치 도달 시 상태 전환
SR-13 파일 업로드	화이트리스트 확장자 + 실제 MIME 확인 + 크기 제한 + 서버가 새 파일명 부여. SVG·HTML 등 스크립트 실행 가능 포

SR	구현 설계
	맷은 명시적으로 제외(확장자 화이트리스트만으로는 SVG가 통과해 저장형 XSS 가능). 가능하면 Pillow로 이미지 재인코딩하여 내장 스크립트 제거. 업로드 디렉터리는 실행권한 없이 정적 서빙
SR-14 경로 조작	사용자 입력을 파일 경로에 직접 사용 금지, secure_filename + 고정 디렉터리
SR-15 로그인 실패 대응	Rate limit(SR-10)과 별개로 연속 실패 시 점진적 지연 또는 일시 계정 잠금
SR-16 전송구간 암호화	운영은 ngrok HTTPS로 HTTP·WebSocket 모두 TLS(WSS) 적용. 로컬 개발은 평문 ws로 동작함을 문서화

2.7 위협 모델 (요약, 자산 기준)

자산	주요 위협	완화책(SR)
계정/비밀번호	무차별 대입, DB 유출, 세션 탈취, 도청	SR-01, SR-06, SR-10, SR-15, SR-16
상품 데이터	IDOR 수정/삭제, XSS, 악성 파일	SR-05, SR-03, SR-13
잔액/송금	음수 송금, 더블스펜딩, 타인 계정 출금	SR-08, SR-05
신고 시스템	자기/반복 신고로 임계치 악용	SR-12
관리자 기능	권한 상승	SR-11
채팅	XSS, 스팸	SR-03, SR-10
DB 전체	SQL Injection	SR-02

2.8 다음 단계

본 설계를 바탕으로 → **03_구현.md** 에서 실제 코드로 구현하고, 각 SR이 코드에서 어떻게 실현되었는지(및 순진한 구현이 왜 취약한지) 기록한다. 이후 **04_테스트.md**(체크리스트+테스트케이스), **05_유지보수.md** 로 이어진다.

3. 시스템 구현

입력 문서: 02_시스템설계.md · 코드: app/ 패키지

3.1 구현 개요

설계(02)의 SR을 실제 코드로 구현했다. Flask 애플리케이션 팩토리 + 기능별 Blueprint 구조이며, DB 접근·인증·검증을 공통 계층으로 중앙화해 보안 통제를 한 곳에서 강제한다.

- 총 코드: app/ 약 1,260 라인(Python wc -l ; SAST 유효 라인 982)
- 테스트: 85개 (전부 통과) — 상세는 04_테스트.md

3.2 모듈 구조

파일	역할	핵심 보안
app/__init__.py	앱 팩토리. CSRF·서버세션·Rate limit·보안헤더·에러핸들러 전역 적용	SR-04,06,09,10
app/config.py	환경변수(.env) 로딩	시크릿 분리
app/db.py	SQLite 접근계층 (query_db / execute_db / transaction)	SR-02, SR-08
app/security.py	argon2 해시, 인증 데코레이터, 입력 검증 유틸	SR-01,05,07
app/uploads.py	이미지 업로드 검증·재인코딩	SR-13,14
app/blueprints/auth.py	회원가입/로그인/로그아웃	SR-01,06,10,15
app/blueprints/products.py	상품 CRUD·검색·이미지 서빙	SR-02,03,05,13,14
app/blueprints/users.py	프로필·마이페이지(소개글·비번)	SR-03,05,07
app/blueprints/chat.py	전체/1:1 채팅(SocketIO)	SR-03,05,10
app/blueprints/reports.py	신고·임계치 자동처리	SR-12, 원자성
app/blueprints/transfer.py	송금(원자적)	SR-08
app/blueprints/admin.py	관리자 기능	SR-11

3.3 공통 보안 통제 (전역)

- **SQL Injection 방지(SR-02):** 모든 DB 접근은 `app/db.py` 의 `query_db / execute_db` 를 통해 ? 파라미터 바인딩만 사용. 문자열 포매팅 쿼리는 코드 전체에 존재하지 않는다.
- **XSS 방지(SR-03):** 서버 렌더링은 Jinja2 자동 이스케이프(`|safe` 미사용). 실시간 채팅은 클라이언트 `chat.js` 에서 `textContent / createTextNode` 로 삽입.
- **CSRF(SR-04):** `CSRFProtect` 전역 적용, 모든 상태변경 폼에 `csrf_token` 히든 필드.
- **세션(SR-06):** 서버 측 세션(Flask-Session), `HttpOnly · SameSite=Lax` , `Secure` (환경 토클), 로그인 시 세션 재발급, 유효 타임아웃.
- **보안 헤더(SR-09):** `X-Content-Type-Options` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Content-Security-Policy` .
- **에러 처리(SR-09):** 전역 에러 핸들러가 사용자에게 일반 메시지, 상세는 서버 로그로만.
- **Rate limit(SR-10):** HTTP는 Flask-Limiter(로그인 등), 채팅은 앱 수준 제한.

3.4 기능별 구현 요점 (SR 실현)

인증 (auth.py)

- 비밀번호는 `argon2-cffi` 로 해시 저장(SR-01). 로그인 검증 실패/성공 메시지를 동일하게 하여 사용자 존재 여부 비노출.
- 로그인·비밀번호 변경 시 **서버측 세션 id(sid)를 실제로 회전**(`regenerate_session`)해 세션 고정 방지(SR-06). `session.clear()` 만으로는 sid가 유지됨을 확인해 교정함(부록 6.17).
- **권한/상태 컬럼(`is_admin · is_dormant`)과 잔액은 회원가입 폼 입력에서 절대 받지 않음** (mass-assignment 차단). `balance` 는 서버 설정값(가입 보너스)만 사용.
- 로그인 실패 rate limit + IP·아이디 기준 점진 잠금(SR-10, SR-15).

상품 (products.py, uploads.py)

- 수정·삭제 전 `product.seller_id == 세션 사용자` 확인, 불일치 시 403(SR-05, IDOR 방지). 삭제는 POST 전용(GET 삭제 금지).
- 검색은 `title LIKE ?` 파라미터 바인딩(SR-02).
- 업로드: 확장자 화이트리스트 → 크기 제한 → Pillow로 실제 이미지 검증 + **재인코딩**(SVG·위장 파일·내장 스크립트 차단, SR-13). 저장 파일명은 서버가 UUID로 부여, 서빙은 `send_from_directory` 로 경로 이탈 차단(SR-14).

마이페이지 (users.py)

- 세션 본인 레코드에만 작용 → 타인 자원 변경 불가. 비밀번호 변경은 **현재 비밀번호 재확인** 후 재해시.

채팅 (chat.py, chat.js)

- 소켓 연결/이벤트마다 세션 인증 확인(미인증 연결 거부). 메시지 길이 검증, 앱 수준 rate limit.
- 1:1 채팅 조회는 `sender_id=나 OR receiver_id=나` 로 제한(SR-05, 타인 대화 열람 차단).
- 구매자↔판매자 흐름: 상품 상세에 “판매자와 1:1 채팅” 버튼, `/chats` 채팅함에서 양측이 대화를 찾아 이어감.

신고 (reports.py)

- 본인 신고 거부, 중복 신고는 DB `UNIQUE` 제약으로 차단(SR-12).
- 신고 INSERT + 카운트 증가 + 임계치 처리를 `BEGIN IMMEDIATE` 트랜잭션으로 원자 처리 → 임계치 도달 시 상품 차단/유저 휴면.

송금 (transfer.py)

- 금액 양수·정수 검증, 자기송금·휴면수신자·부존재 차단.
- 조건부 `UPDATE( balance = balance - ? WHERE id=? AND balance >= ? )` + affected row 확인 + `BEGIN IMMEDIATE` 로 잔액 음수화·더블스펜딩 방지(SR-08). DB에도 `CHECK(balance>=0)` 최후 방어선.

관리자 (admin.py)

- 전 라우트 `@admin_required`. 비관리자 접근 403(SR-11). 상품 강제 삭제/차단해제, 유저 휴면/복구.

3.5 데이터 무결성 (DB 제약)

`app/schema.sql` 에 애플리케이션 검증과 별개의 최후 방어선을 둔다: `PRAGMA foreign_keys=ON, NOT NULL, user.balance CHECK(>=0), transfers.amount CHECK(>0), transfers 의 sender_id<>receiver_id, report 의 UNIQUE(reporter_id,target_type,target_id) .`

→ 실제 발견·수정한 보안 약점의 Before/After는 `06_보안약점_before_after.md` 참조.

4. 테스트 (체크리스트 & 테스트 케이스)

슬라이드 방법론의 “체크리스트 작성 및 확인” + “실제 테스트” 단계. 구현된 각 부분이 요구사항·보안요소를 만족하는지 체크리스트로 점검하고, 자동화 테스트로 검증한다.

4.1 자동화 테스트 개요

- 프레임워크: `pytest` + Flask/Flask-SocketIO 테스트 클라이언트
- 위치: `tests/`
- 결과: 85개 전부 통과
- 실행: `pytest -q` (환경 세팅은 루트 README.md 참조)

테스트 파일	개수	대상
test_auth.py	13	회원가입/로그인/로그아웃, SQLi·mass-assignment·휴면·타이밍 열거
test_products.py	17	상품 CRUD·검색, IDOR·XSS·업로드·음수가격·이미지 파일 정리·압축폭탄
test_users.py	8	프로필/마이페이지, 비번 재확인·XSS
test_chat.py	13	소켓 인증·휴면차단·길이·rate limit·1:1 IDOR·XSS·채팅함
test_reports.py	10	신고·임계치, 자기/중복 신고, 관리자 DoS 방어
test_transfer.py	9	송금, 음수/자기/부족/휴면·더블스펜딩
test_admin.py	8	관리자 권한·동작
test_security.py	4	보안헤더·세션쿠키·CSRF
test_requirements.py	3	최소요구 보완분(내 상품 관리·1:1 채팅 UI)

각 테스트는 정상 케이스 + 공격/오용 케이스를 쌍으로 둔다(정상 동작하면서 공격은 막히는지).

4.2 보안 체크리스트 (슬라이드 31쪽 기준)

구분	체크 항목	구현	검증
회원가입/프로필	서버측 입력 검증	security.validate_*	test_auth, test_products
	CSRF 보호	CSRFProtect 전역	test_security
	비밀번호 안전 저장	argon2id + salt	test_auth
	세션 쿠키 설정	HttpOnly·SameSite·Secure(토글)	test_security
	세션 만료/재인증	서버세션 + 타임아웃, 비번변경 재확인	test_users
	로그인 실패 방어	rate limit + 점진 잠금	auth.py (SR-10,15)
	오류 메시지 정보노출 방지	전역 에러 핸들러	test_security(headers)
상품 등록/관리	폼 입력 검증	title/price/desc 검증	test_products
	XSS 방어	Jinja 이스케이프	test_products
	인증 사용자만 등록	@login_required	test_products
	소유자 확인	seller_id 검증	test_products (IDOR 403)
	데이터 무결성	DB 제약(NOT NULL/FK/CHECK)	schema.sql
실시간 채팅	메시지 내용 검증	길이 제한	test_chat
	사용자 인증	소켓 connect 인증	test_chat
	Rate Limiting	앱 수준 제한	test_chat
	연결 암호화(WSS/TLS)	ngrok HTTPS	운영 구성
신고	입력 검증	reason/target 검증	test_reports
	인증 사용자만	@login_required	test_reports
	데이터 무결성/로그	UNIQUE + 원자 트랜잭션	test_reports
	신고 남용 방지	자기/중복 신고 차단	test_reports
송금	금액/수신자 검증	양수·존재·활성	test_transfer
	원자성(더블스펜딩)	조건부 UPDATE + 트랜잭션	test_transfer
관리자	권한 통제	@admin_required	test_admin (403)
의존성	취약점 점검	pip-audit	05_유지보수 (0건)

4.3 대표 공격 시나리오 테스트 (발취)

공격	테스트	기대 결과
인증 우회 ' OR '1'='1'--	test_login_sql_injection_blocked	401, 세션 없음
관리자 승격 is_admin=1	test_register_ignores_privilege_fields	is_admin=0
타인 상품 삭제	test_delete_others_product_forbidden	403, 데이터 불변
저장형 XSS <script>	test_stored_xss_escaped 등	이스케이프 출력
동시 송금 더블스펜딩	test_no_double_spend_under_concurrency	1건만 성공, 잔액 0
음수 송금 -500	test_transfer_negative_amount_rejected	400
SVG 업로드	test_upload_svg_rejected	400
중복 신고	test_duplicate_report_rejected	카운트 미증가
비관리자 관리페이지	test_non_admin_forbidden	403
CSRF 토큰 없는 POST	test_post_without_csrf_token_rejected	400

4.4 실행 결과

```
$ pytest -q
```

```
..... [100%]
85 passed
```

전 항목 통과. 실패 케이스 재현 및 회귀는 `pytest tests/<파일>::<테스트>` 로 개별 실행 가능.

4.5 정적 보안 분석 (SAST)

슬라이드가 언급한 “시큐어 코딩 점검 툴”에 대응해, 두 계층에서 자동 보안 점검을 수행한다.

계층	도구	실행	결과
소스코드	bandit (Python SAST)	<code>bandit -r app/</code>	High 0 · Medium 0 · Low 0 (982 라인)
의존성	pip-audit	<code>pip-audit -r requirements.txt</code>	취약점 0건

- bandit: SQL 문자열 조립, 하드코딩 시크릿, 취약 해시, 광범위 예외 처리 등 위험 패턴을 정적 탐지 → 0건.
- 초기엔 타이밍 평준화 코드의 `except Exception: pass (B110)`가 Low로 검출 → 특정 argon2 예외만 잡도록 수정해 해소.

→ 코드·의존성 양쪽에서 알려진 위험 패턴이 남지 않았음을 도구로 확인.

5. 유지보수

배포 후 지속적으로 안전한 상태를 유지하기 위한 절차. 슬라이드의 “유지보수” 단계.

5.1 의존성 취약점 관리 (실제 수행)

`pip-audit` 로 서드파티 라이브러리의 알려진 취약점(CVE)을 점검한다.

Before — 초기 의존성 점검 결과: 9건 발견

```
Found 9 known vulnerabilities in 3 packages
flask          3.1.0    CVE-2025-47278, CVE-2026-27205
python-dotenv  1.0.1    CVE-2026-28684
pillow         11.0.0   CVE-2026-25990, CVE-2026-40192, CVE-2026-42310,
                                CVE-2026-42311, PYSEC-2026-165 (2)
```

조치 — 안전 버전으로 업그레이드 | 패키지 | Before | After | |—|—|—| Flask | 3.1.0 | 3.1.3 | | python-dotenv | 1.0.1 | 1.2.2 | | Pillow | 11.0.0 | 12.3.0 | | Flask-SocketIO | 5.4.1 | 5.6.1 (Flask 3.1.x 호환성) |

After — 재점검 결과: 0건

```
$ pip-audit -r requirements.txt
No known vulnerabilities found
```

업그레이드 후 전체 테스트 85개 재실행하여 회귀 없음을 확인했다.

정책: 배포 전 정기적으로 `pip-audit` 를 실행하고, `requirements.txt` 의 버전을 고정(pin)하여 재현 가능한 빌드를 유지한다.

5.2 시크릿 관리

- `SECRET_KEY`, ngrok authtoken, 관리자 초기 비밀번호 등은 코드/레포에 하드코딩하지 않는다.
- 실제 값은 `.env` 에 두고 `.gitignore` 로 커밋에서 제외. 템플릿은 `.env.example` 로 공유.

- SECRET_KEY 는 secrets.token_hex(32) 로 무작위 생성.
- 유출 시 대응: SECRET_KEY 재발급(전 세션 무효화), ngrok 토큰 재발급, 관리자 비밀번호 변경.

5.3 운영 배포 설정

- FLASK_ENV=production → DEBUG=False (스택트레이스 노출·Werkzeug 콘솔 차단, SR-09).
- SESSION_COOKIE_SECURE=1 (HTTPS/ngrok).
- ngrok HTTPS 터널로 HTTP·WebSocket(WSS) 모두 TLS 적용.
- 개발 서버(Werkzeug) 대신 운영에서는 WSGI 서버 사용을 권장.

5.4 로깅 & 모니터링

- 서버 오류는 전역 에러 핸들러가 app.logger.exception 으로 서버 로그에만 기록(사용자에게 일반 메시지).
- 권장 확장: 로그인 실패/신고/송금 등 보안 이벤트 감사 로그, 이상 징후 알림.

5.5 백업 & 데이터 관리

- SQLite DB 파일(instance/market.db)과 업로드 디렉터리(uploads/)를 정기 백업.
- 업로드 파일은 재인코딩되어 저장되므로 원본 악성 페이로드가 남지 않는다.

5.6 사고 대응 절차 (요약)

1. 탐지: 비정상 로그인/송금/신고 급증 감지.
2. 격리: 해당 계정 휴면 처리(관리자), 필요 시 서비스 중단.
3. 원인 분석: 로그·DB 상태 점검.
4. 복구: 취약점 수정 → 테스트 → 재배포, 필요 시 SECRET_KEY 재발급.
5. 재발 방지: 회귀 테스트 추가, 체크리스트 갱신.

5.7 유지보수 관점의 개선 여지 (알려진 한계)

- Rate limit 저장소는 설정 가능: 기본은 인메모리(단일 프로세스 ngrok 배포에 적합)이나, RATELIMIT_STORAGE_URI=redis://... 로 지정하면 다중 워커에서도 공유된다. 로그인 실패

패 잠금·채팅 rate limit의 커스텀 인메모리 상태는 단일 프로세스 전제이며, 수평 확장 시 공유 저장소로 이전 권장.

- 채팅 클라이언트(socket.io)는 **자체 호스팅**(`static/js/socket.io.min.js`)으로 전환 → 외부 CDN 의존 제거, CSP `script-src 'self'` 로 강화, 폐쇄망에서도 동작.
- 세션 저장소가 파일시스템 기반 → 수평 확장 시 공유 저장소 필요.
- Flask-Session 파일시스템 백엔드는 향후 API 변경 예정(현재 동작에는 문제 없음).

이러한 한계는 과제 규모에서는 허용되나, 실제 서비스 확장 시 우선 개선 대상으로 문서화해 둔다.

6. 발견한 보안 약점과 수정 (Before / After)

과제 핵심 요구사항인 “확인한 보안 약점이 무엇이고 어떻게 변경했는지”를 정리한다. 각 항목은 **취약 코드(Before) → 공격 시나리오 → 수정(After) → 검증**으로 구성한다. “취약한 순진한 구현”은 개발 중 실제로 경계한 안티패턴이며, 최종 코드는 모두 After 상태다. 검증은 `tests/` 의 실제 테스트로 자동화되어 있다(총 85개 통과).

- **6.1~6.12:** 설계 단계에서 선제 제거한 표준 취약점. Before(취약 코드)가 가설이 아님은 실증 스크립트로 확인 — 예: `보안실증/demo_sqli.py` 실행 시 취약 코드는 `' OR '1'='1'--` 로 **인증 우회 성공**, 수정 코드(파라미터 바인딩)는 **차단**.
- **6.13~6.17:** 개발·리뷰 중 실제로 발견해 수정한 사례(의존성 CVE 9건, 다중 에이전트 패널이 잡은 세션 고정 등).

6.1 SQL Injection — 인증 우회 (SR-02)

Before (취약)

```
# 문자열 포매팅으로 쿼리를 조립하면 인젝션에 노출
user = db.execute(
    f"SELECT * FROM user WHERE username = '{username}'"
).fetchone()
```

공격: 아이디에 `' OR '1'='1'--` 입력 → 쿼리가 항상 참이 되어 인증 우회.

After (수정) — `app/blueprints/auth.py`

```
user = query_db("SELECT * FROM user WHERE username = ?", (username,),
    one=True)
```

파라미터 바인딩으로 입력이 데이터로만 취급된다. 프로젝트 전체가 이 규칙을 따른다.

검증: `test_auth.py::test_login_sql_injection_blocked` — `' OR '1'='1'--` 로그인 시 401, 세션 미생성.

6.2 비밀번호 평문 저장 (SR-01)

Before

```
execute_db("INSERT INTO user (username, password) VALUES (?, ?)", (username, password))
```

공격: DB 유출 시 전 계정 비밀번호가 즉시 노출.

After — `app/security.py`, `auth.py`

```
from argon2 import PasswordHasher
_ph = PasswordHasher()
# 저장
execute_db("INSERT INTO user (... , password_hash, ...) VALUES (... , ?, ...)",
           (hash_password(pw),))
# 검증
_ph.verify(stored_hash, pw)
```

argon2id + salt 자동 적용. 평문·단순 해시(MD5/SHA)를 쓰지 않는다.

검증: `test_auth.py::test_register_success_and_password_hashed` — 저장값이 `$argon2`로 시작, 평문 미포함.

6.3 권한 상승 — Mass Assignment (SR-11)

Before

```
# 폼 전체를 그대로 저장 → is_admin 을 폼에 넣으면 관리자가 됨
cols = ",".join(request.form.keys())
```

공격: 회원가입 요청에 `is_admin=1` (또는 `balance=999999`) 필드를 추가 → 누구나 관리자/부자.

After — `app/blueprints/auth.py`

```
execute_db(
    "INSERT INTO user (id, username, display_name, password_hash, balance,
        created_at) VALUES (?, ?, ?, ?, ?, ?)",
    (uuid, username, display_name, hash_password(pw), REGISTER_BONUS,
        now_iso()),
) # is_admin/is_dormant 는 삽입 대상에서 제외(기본값), balance 는 서버 상수만 사용
```

검증: `test_auth.py::test_register_ignores_privilege_fields` —
`is_admin=1&balance=999999` 주입해도 무시됨(핵심: 사용자 입력이 특권/상태 컬럼에 반영되지 않음). 테스트는 설정 `REGISTER_BONUS=0` 기준이라 `balance=0`; 운영 기본 가입 보너스는 1000이며 이 역시 서버 상수만 사용.

6.4 저장형 XSS — 상품/소개글/채팅 (SR-03)

Before

```
<div>{{ product.description | safe }}</div>  <!-- 이스케이프 해제 -->
```

```
li.innerHTML = sender + ": " + msg;  // 채팅을 innerHTML 로 삽입
```

공격: 상품 설명/소개글/채팅에 `<script>alert(1)</script>` 저장 → 열람자 브라우저에서 실행.

After - 서버 렌더링: Jinja2 자동 이스케이프 유지, `|safe` 미사용. - 채팅(클라이언트 렌더):

`app/static/js/chat.js`

```
li.appendChild(document.createTextNode(content));  // textContent → HTML 미해석
```

검증: `test_products.py::test_stored_xss_escaped`,
`test_users.py::test_bio_xss_escaped`, `test_chat.py::test_chat_history_escaped` —
원본 태그 미출력, `<script>` 로 이스케이프.

6.5 IDOR — 타인 상품 수정/삭제 (SR-05)

Before

```
@bp.route("/products/<id>/delete", methods=["POST"])  
@login_required  
def delete(id):  
    execute_db("DELETE FROM product WHERE id = ?", (id,))  # 소유자 확인 없음
```

공격: 남의 `product_id` 로 삭제/수정 요청 → 그대로 실행. (UUID여도 방어 아님)

After — `app/blueprints/products.py`

```
p = query_db("SELECT * FROM product WHERE id = ?", (id,), one=True)
if p is None: abort(404)
if p["seller_id"] != current_user()["id"]: abort(403) # 소유자 검증
```

검증: `test_products.py::test_edit_others_product_forbidden`,
`test_delete_others_product_forbidden` — 타인 자원 조작 시 403, 데이터 불변. 채팅 1:1도 동일 원리(`test_chat.py::test_private_chat_idor_blocked`).

6.6 송금 경쟁 조건 — 더블스펜딩 (SR-08)

Before

```
bal = query_db("SELECT balance FROM user WHERE id=?", (me,), one=True)
["balance"]
if bal >= amount: # 확인
    execute_db("UPDATE user SET balance=? WHERE id=?", (bal - amount, me)) # 갱신
```

공격: 잔액 100인 계정이 동시에 100씩 두 번 송금 → 두 요청 모두 확인 통과 후 갱신 → 잔액 음수 (더블스펜딩).

After — `app/blueprints/transfer.py` + `app/db.py::transaction`

```
with transaction() as db: # BEGIN IMMEDIATE
    cur = db.execute(
        "UPDATE user SET balance = balance - ? WHERE id = ? AND balance >= ?",
        (amount, me, amount))
    if cur.rowcount != 1: # 잔액 부족이면 0행 → 실패
        raise _Insufficient()
    db.execute("UPDATE user SET balance = balance + ? WHERE id = ?", (amount, receiver))
```

조건부 UPDATE + 트랜잭션 잠금으로 원자성 확보. DB에도 `CHECK(balance>=0)`.

검증(2단): - DB 계층: `test_transfer.py::test_no_double_spend_under_concurrency` — 조건부 UPDATE를 별도 연결 스레드 2개로 실행해 1건만 성공·잔액 0. - 엔드포인트 계층: 보안실 `중/demo_concurrency.py` — 스레드 2개가 실제 `/transfer` 라우트를 동시 호출(라우트+세션+DB 전 경로). 잔액 1회분에서 1건만 302 성공·1건 400 거부, 송신자 잔액 0, 거래기록 정확히 1건 확인. - 정상/초과: `test_transfer_success`, `test_transfer_over_balance_rejected`.

6.7 잘못된 송금 입력 — 음수/자기송금 (SR-08)

Before: 금액 부호·수신자 검증 없음 → `amount=-500` 송금 시 오히려 잔액 증가, 자기 자신 송금 허용.

After — 서버측 검증 + DB 제약

```
aerr, amount = validate_int(form["amount"], "송금 금액", min_v=1,
                             max_v=1_000_000_000)
if receiver["id"] == me["id"]: errors.append("자기 자신에게는 송금할 수 없습니다.")
```

DB: `transfers.amount CHECK(amount > 0)`, `CHECK(sender_id <> receiver_id)`.

검증: `test_transfer.py::test_transfer_negative_amount_rejected`,
`test_transfer_zero_amount_rejected`, `test_transfer_to_self_rejected`.

6.8 악성 파일 업로드 — SVG/위장 이미지 (SR-13)

Before

```
file.save(os.path.join(UPLOAD_DIR, file.filename)) # 확장자·내용 검증 없이 원본 저장
```

공격: `evil.svg` (내부 `<script>`) 업로드 → 저장형 XSS. 또는 `evil.png` 로 위장한 실행 파일 업로드.

After — `app/uploads.py`

```
if ext not in {"jpg", "jpeg", "png", "gif", "webp"}: raise UploadError(...) #
    SVG 제외
probe = Image.open(BytesIO(data)); probe.verify() # 실제
    이미지 검증
img = Image.open(BytesIO(data)); img.save(path, format=img.format) # 재인
    코딩(스크립트 제거)
```

저장 파일명은 서버가 UUID로 부여, 서빙은 `send_from_directory` (경로 이탈 차단).

검증: `test_products.py::test_upload_svg_rejected`,
`test_upload_fake_image_rejected` — 각각 400.

6.9 신고 남용 — 자기/중복 신고 (SR-12)

Before: 신고에 아무 제한이 없어 한 사람이 반복 신고로 임계치를 채워 타인을 휴면/차단시킬 수 있음.

After — `app/blueprints/reports.py` + `schema.sql`

```
if target_id == me["id"]: errors.append("자기 자신은 신고할 수 없습니다.")
# INSERT 시 UNIQUE(reporter_id, target_type, target_id) 위반 → 중복 신고 차단
```

신고 카운트 증가·임계치 처리는 `BEGIN IMMEDIATE` 트랜잭션으로 원자 처리.

검증: `test_reports.py::test_self_report_user_rejected`,
`test_duplicate_report_rejected` (중복은 카운트 미증가),
`test_report_own_product_rejected`.

6.10 CSRF — 상태변경 위조 (SR-04)

Before: 토큰 없이 POST 처리 → 공격자 사이트가 피해자 브라우저로 비밀번호 변경·송금·상품삭제 요청을 위조.

After — `app/__init__.py`

```
csrf = CSRFProtect(); csrf.init_app(app) # 전역
```

모든 폼에 `<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">`.

검증: `test_security.py::test_post_without_csrf_token_rejected` (토큰 없음 400) /
`test_post_with_csrf_token_accepted` (정상 302).

6.11 세션·전송 보안 (SR-06, SR-16)

Before: 클라이언트 쿠키 세션(휴면 처리해도 즉시 무효화 불가), 쿠키 플래그 미설정, 평문 전송.

After - 서버 측 세션(Flask-Session) → 휴면/로그아웃 시 서버가 즉시 무효화. -

`SESSION_COOKIE_HTTPONLY=True`, `SAMESITE=Lax`, `SECURE` (HTTPS 토글), 유희 타임아웃. - 운
영은 ngrok HTTPS로 HTTP·WebSocket(WSS) 모두 TLS.

검증: `test_security.py::test_session_cookie_httponly_samesite`,
`test_reports.py::test_dormant_user_cannot_login_after_reports`.

6.12 정보 노출 & 무차별 대입 (SR-09, SR-10, SR-15)

- **에러 메시지:** 전역 에러 핸들러가 스택트레이스/쿼리 대신 일반 메시지만 노출 (`app/__init__.py`). 운영 `DEBUG=False` .
- **로그인 보호:** Flask-Limiter(분당 제한) + 연속 실패 시 점진 잠금(`auth.py`).

검증: `test_security.py::test_security_headers_present` , 보안 헤더 4종 적용.

6.13 의존성 취약점 (유지보수, SR 상시)

`pip-audit` 로 초기 의존성에서 **9건의 알려진 취약점**(`flask·python-dotenv·pillow`) 발견 → 안전 버전으로 업그레이드(`Flask 3.1.3, python-dotenv 1.2.2, Pillow 12.3.0, Flask-SocketIO 5.6.1`) → **재점검 0건**. 상세는 `05_유지보수.md`.

6.14 신고 기반 관리자 계정 잠금 — DoS (독립 감사에서 발견)

Before: 신고 임계치 자동 휴면 로직에 관리자 예외가 없어, 일반 유저 여러 명이 담합하여 관리자 계정을 신고하면 관리자가 휴면 처리되어 로그인이 차단됨(서비스 마비).

공격: 서로 다른 계정 N개(임계치)로 관리자 `user_id` 를 신고 → `is_dormant=1` → 관리자 로그인 불가.

After — `app/blueprints/reports.py::_apply_threshold`

```
row = db.execute("SELECT report_count, is_admin FROM user WHERE id = ?",
                 (target_id,)).fetchone()
if row["report_count"] >= threshold and not row["is_admin"]: # 관리자는 자동 휴면 제외
    db.execute("UPDATE user SET is_dormant = 1 WHERE id = ?", (target_id,))
```

신고 기록·카운트는 남기되(감사 추적), 관리자 계정은 자동 휴면 대상에서 제외한다.

검증: `test_reports.py::test_admin_not_auto_dormant_by_reports` — 임계치까지 신고해도 관리자 `is_dormant=0` .

6.15 워커 풀 다중 리뷰 반영 (2차 하드닝)

17개 리뷰 워커(기능·개발자 2렌즈 × 8모듈 + 종합)로 재감사해 발견·수정한 항목:

- **CSP vs 인라인 핸들러 충돌:** `script-src` 에 `unsafe-inline` 이 없어 삭제확인
`onsubmit="confirm()"` 이 CSP에 차단 → 확인창 없이 삭제되던 문제. 인라인 제거 후 외부
`static/js/confirm.js` (`data-confirm` 속성)로 대체, CSP 무결성 유지.
(`test_products.py` , 소유자 페이지 렌더 검증)
- **소켓 계층 인가 우회:** SocketIO 핸들러가 휴면 사용자를 검증하지 않아 제재된 사용자가 채팅
지속 가능 → `connect/메시지` 핸들러에 `is_dormant` 검증 추가.
(`test_chat.py::test_dormant_user_socket_rejected`)
- **업로드 파일 라이프사이클:** 상품 삭제 시 이미지 파일 미삭제(고아), 수정 시 이미지 교체 불가,
폼 검증 실패 시 이미지 고아 → 삭제 시 파일 제거, 수정 시 교체+옛 파일 정리, 검증 순서 교정.
(`test_delete_removes_image_file` , `test_edit_replaces_image_and_removes_old`)
- **비밀번호 변경 후 세션 재발급:** 변경 후 세션을 재발급해 기존(탈취) 세션 무효화.
(`test_password_change_keeps_session`)
- (부가) SQLite `PRAGMA busy_timeout=5000` 명시로 동시 쓰기 시 즉시 실패 대신 대기.

참고: 위커가 제기한 “동시 송금 시 `database is locked 500`”은 실측 결과 재
현되지 않음(sqlite3 기본 `timeout`이 대기 처리) — 과대보고로 판단하고 별도 수
정 없이 `busy_timeout` 만 명시.

6.16 방어적 하드닝 (테스터 관점 선제 조치)

침투 테스트에서 노릴 수 있는 잔여 표면을 선제적으로 제거:

- **로그인 타이밍 기반 사용자 열거:** 존재하지 않는 아이디는 `argon2` 검증을 건너뛰어 응답이 빨
라짐 → 유효 아이디 추측 가능. 아이디가 없을 때도 더미 해시로 `argon2` 검증을 수행해 응답
시간 차 제거. (`test_auth.py::test_login_timing_enumeration_mitigated`)
 - **이미지 압축폭탄(DoS):** 작은 파일이지만 초대형 해상도인 이미지로 메모리 고갈 유발 →
25MP 픽셀 상한 + `DecompressionBombError` 처리로 차단.
(`test_products.py::test_upload_oversized_image_rejected`)
 - **회원가입 동시성 UNIQUE 충돌:** 동시 가입 시 `IntegrityError` 로 500 노출 → 친절한 400
처리.
 - **로그인 잠금 메모리 남용:** 공격자가 다양한 아이디/IP로 인메모리 잠금 `dict`를 무한 증식 → 만
료 항목 자동 정리.
-

6.17 세션 고정(Session Fixation) — PM 리뷰 패널이 발견

Before: 로그인 시 `session.clear()` 후 재대입만 하여, Flask-Session의 서버측 세션 id(sid)가 **회전되지 않음**. 설계 주석은 “세션 재발급(SR-06)”이라 했으나 실제로는 미작동 → 공격자가 피해자 브라우저에 심은 sid가 로그인 후에도 유지되어 세션 고정→계정 탈취 가능.

공격(라이브 재현): curl로 로그인 전/후 `session` 쿠키 값이 **동일함**을 확인 → 취약점 확정.

After — `app/security.py::regenerate_session`, `auth.py`, `users.py`, `schema.sql`

```
def regenerate_session():
    session.clear()
    if hasattr(session, "sid"):
        session.sid = secrets.token_urlsafe(32) # 서버측 sid 실제 회전
    session.modified = True
```

- 로그인·비밀번호 변경 시 sid를 실제로 회전(재현 결과 로그인 전/후 sid 상이).
- `user.session_epoch` 도입 → 비밀번호 변경 시 증가시켜 **다른 기기의 기존 세션 전부 무효화**(`current_user` 가 epoch 불일치 세션을 로그아웃 처리).

검증: `test_auth.py::test_login_rotates_session_cookie` (전후 sid 상이),
`test_password_change_invalidates_old_sessions` (옛 세션 무효), 라이브 재현.

이 항목은 다중 에이전트 PM 리뷰 패널이 “보고서가 방어된다고 단언했으나 실제 미작동”으로 지적 → 재현으로 확인 후 수정한 사례다. (허위 방어 주장을 실증으로 교정)

요약 표

#	약점	유형(KISA)	수정	검증 테스트
6.1	SQL Injection	입력검증	파라미터 바인딩	test_auth
6.2	평문 비밀번호	보안기능	argon2 해시	test_auth
6.3	Mass Assignment	캡슐화	컬럼 화이트리스트	test_auth
6.4	저장형 XSS	입력검증	이스케이프/textContent	test_products/users/chat
6.5	IDOR	캡슐화	소유자 검증	test_products/chat
6.6	송금 경쟁조건	시간·상태	조건부 UPDATE+트랜잭션	test_transfer
6.7	음수/자기송금	입력검증	검증+DB CHECK	test_transfer

#	약점	유형(KISA)	수정	검증 테스트
6.8	악성 업로드	입력검증	화이트리스트+재인코딩	test_products
6.9	신고 남용	시간·상태	자기/중복 차단	test_reports
6.10	CSRF	보안기능	CSRFProtect	test_security
6.11	세션/전송	보안기능	서버세션·쿠키·TLS	test_security/reports
6.12	정보노출/무차별	에러·시간상태	에러핸들러·rate limit	test_security
6.13	의존성 취약점	API오용	버전 업그레이드	pip-audit 0건
6.14	관리자 신고-DoS	시간·상태	관리자 자동휴면 예외	test_reports
6.15a	CSP-인라인 충돌	보안기능	외부 JS(data-confirm)	test_products
6.15b	소켓 인가 우회	캡슐화	소켓 휴면 검증	test_chat
6.15c	업로드 파일 고아	코드오류	삭제/교체 시 파일 정리	test_products
6.15d	비번변경 세션	보안기능	세션 재발급	test_products
6.16a	로그인 타이밍 열거	보안기능	더미 argon2 검증	test_auth
6.16b	이미지 압축폭탄	입력검증	픽셀 상한(25MP)	test_products
6.16c	가입 UNIQUE 충돌 500	에러처리	IntegrityError 400 처리	—
6.16d	잠금 메모리 남용	시간·상태	만료 항목 정리	—
6.17	세션 고정	보안기능	sid 회전 + epoch 무효화	test_auth

7. AI 도구 활용

과제가 강조한 “ChatGPT·Copilot·Cursor·Claude 등 AI 도구를 최대한 적극적으로 활용할 것”에 대한 기록. 단순 코드 생성을 넘어, **AI를 개발·검증·적대적 리뷰 전 과정에 활용**하고 그 결과를 사람이 직접 재현·검증했다.

7.1 활용 방식

단계	AI 활용
구현	Claude(Claude Code)로 요구사항 분석·설계·전체 코드 작성
정적 점검	<code>bandit</code> (SAST)· <code>pip-audit</code> 로 코드·의존성 자동 진단
적대적 리뷰	다중 에이전트 워크플로우로 코드를 여러 관점에서 교차 검토
검증	AI 주장을 믿지 않고 사람이 직접 재현·테스트 로 확인(환각 방지)

7.2 다중 에이전트 적대적 리뷰 (핵심)

단일 AI의 자기확인 편향을 피하기 위해, **독립 컨텍스트의 여러 리뷰 에이전트**를 병렬로 돌려 교차 검증했다.

- **모듈 리뷰**: 8개 모듈 × (기능 리뷰어 + 개발자 리뷰어) 2렌즈 병렬 + 종합.
- **PM 리뷰 패널**: 보안 감사·요구사항 감사·코드 품질·보고서 정합성 + PM 종합.
- **아키텍처**: Scatter-Gather(팬아웃→팬인). 오케스트레이션은 코드로 결정(재현 가능), 판단은 에이전트가 수행.

중요한 건 에이전트 수가 아니라 **결과다** — 이 방식이 사람이 놓친 세션 고정 취약점을 실제로 잡았다는 것.

7.3 AI가 실제로 잡아낸 취약점 (사람이 놓친 것)

다중 에이전트 리뷰가 없었다면 놓쳤을 실제 결함들:

항목	발견 경로	내용
세션 고정 (6.17)	PM 보안 패널	보고서가 “세션 재발급으로 방어”라 단언했으나 실제로는 sid 미회전. 라이브 재현으로 확인 후 수정.
관리자 신고 DoS (6.14)	17-워커 리뷰	담합 신고로 관리자 휴면 잠금 가능 → 관리자 예외 처리
CSP-인라인 충돌 (6.15a)	17-워커 리뷰	삭제확인 인라인 핸들러가 CSP에 차단 → 외부 JS로 교체
소켓 인가 우회 (6.15b)	17-워커 리뷰	휴면 사용자가 소켓으로 채팅 지속 가능 → 소켓 휴면 검증

특히 세션 고정은 “내가 방어했다고 문서에 쓴 통제가 실제로는 작동하지 않던” 사례로, 다중 검증의 가치를 보여준다.

7.4 환각 방지 원칙

AI 결과를 그대로 신뢰하지 않았다: - 모든 취약점 주장을 직접 재현(예: 세션 고정은 curl로 로그인 전후 sid 동일함을 확인)했다. - 리뷰가 제기한 “동시 송금 500” 같은 항목은 실측 결과 재현되지 않아 반영하지 않았다(과대보고 판별). - 최종 상태는 85개 자동화 테스트 + SAST 0 + 의존성 0으로 기계 검증했다.

7.5 결론

AI를 “코드를 대신 써주는 도구”가 아니라 **적대적 검증 파트너**로 활용했고, 그 발견을 사람이 재현·수정·회귀검증하는 루프를 돌렸다. 시큐어 코딩에서 보안은 결국 사람이 최종 판단하는 영역임을 실천적으로 보여준다.

